

QGC UI 스크립팅 — 전역 녹화 · 재생 · 시나리오

QGroundControl + OpenUxAS · EventBroadcaster 기반 · 2026-07 · Add3Dmap

QGC의 모든 UI를 외부에서 녹화·재생·자동조작하기 위한 시스템 설명서다. 사람이 버튼을 누르는 것과 동일하게, 외부 프로그램이 시간·조건에 따라 QGC 전체를 처음부터 끝까지 구동한다.

1. 구성 한눈에



포트	방향	내용
45678	QGC → 외부	모든 UI 이벤트(<code>ui</code>), 패널(<code>uxas-ui</code>), 텔레메트리(<code>telemetry</code>), 컨트롤 목록(<code>ui_registry</code>)
45679	외부 → QGC	UI 조작 명령, 고수준 명령(웨이포인트 이동 등)

2. 전역 UI 후킹 (QGC 내부)

QGC의 거의 모든 화면은 6개 베이스 컨트롤로 만들어진다. 그 베이스에 후킹을 넣어 앱 전체가 자동으로 녹화·재생 가능해진다. (부팅 직후 실측 약 160개 컨트롤 자동 등록.)

컨트롤	타입	녹화 이벤트	재생 동작
QGCButton	<code>button</code>	클릭	<code>clicked()</code> 발생
QGCCheckBox	<code>checkbox</code>	체크/해제	<code>checked</code> 설정 + <code>clicked()</code>
QGCRadioButton	<code>radio</code>	선택	<code>checked</code> 설정 + <code>clicked()</code>
QGCCoboBox	<code>combobox</code>	항목 선택	<code>currentIndex</code> 설정 + <code>activated()</code>
QGCTextField	<code>textfield</code>	입력 완료	<code>text</code> 설정 + <code>editingFinished()</code>
QGCSlider	<code>slider</code>	값 이동	<code>value</code> 설정 + <code>moved()</code>

검증 완료 — 외부에서 `{"action": "ui", "id": "checkbox:Camera coverage (green)", "value": true}` 전송 → QGC가 그 체크박스를 실제로 눌러 패널 핸들러 실행 확인.

컨트롤 id 규칙

- 명시적 `uiId` 가 있으면 그것 (가장 안정적)
- 없으면 `objectName`
- 없으면 `<타입>:<텍스트/라벨>` — 예: `button:Arm` , `checkbox:River`
- 같은 라벨이 여러이면 `#1` , `#2` 접미사
- 콤보박스는 텍스트가 없어 현재 선택 텍스트로 라벨 (예 `combobox:Multi-Rotor`). 시나리오 핵심 콤보엔 명시적 `uiId` 부여됨:
`ux_sensor_combo` , `ux_region_shape_combo` , `ux_river_mode_combo`

주의 — id는 UI 텍스트 기반이라 화면 문구가 바뀌면 id도 바뀐다. 뷰가 로드되어 있어야 그 컨트롤이 등록된다(예: Plan 뷰 버튼은 Plan 뷰를 한 번 열어야 목록에 뜬).

3. UI Automation Console `ui_scenario_studio.py`

EventBroadcaster 도구(모니터·녹화·재생·시나리오)를 탭 하나의 올인원 GUI로 통합한 프로그램이다.

```
# GUI (탭: Monitor/Record · Scenario · Replay · Stack)
python3 ui_scenario_studio.py

# 헤드리스 실행
python3 ui_scenario_studio.py --run scenario.json # 시나리오
python3 ui_scenario_studio.py --replay session.jsonl # 녹화 재생
```

왼쪽(항상 표시): ① **UI controls** — QGC가 방송하는 전 컨트롤 팔레트(검색·Refresh= `__dump__`), ② **Live events** — 지금 QGC에서 누른 것이 실시간으로 뜨고 더블클릭하면 시나리오 스텝으로 추가, ③ **Telemetry** — 차량 상태 실시간.

탭	기능
Monitor / Record	전 채널 이벤트 스트림 + 필터, ● Record로 <code>.jsonl</code> 녹화
Scenario	스텝 작성(팔레트/라이브피드→스텝), 트리거(시간/조건), Run/Save/Load, 녹화 Import. 표는 #·When·Do로 사람이 읽는 라벨 표시 (우상단 <code>raw JSON view</code> 로 토글)
Replay	녹화 <code>.jsonl</code> 불러와 원래 타이밍대로 재생(rate·필터·dry-run)
Stack	UxAS·PX4 함대·브리지·리스너·QGC 버튼으로 기동/종료

이전의 `eb_monitor(_gui) · eb_replay · eb_scenario` 기능이 이 콘솔에 흡수됨. Scenario 표의 "Do" 칸은 `click button:Arm · Start Mission · takeoff 80 m · → waypoint 3` 처럼 읽기 쉽게 표시된다.

4. 시나리오 파일 형식

```
{
  "name": "SAR 전체 임무",
  "steps": [
    {"wait": 2, "cmd": {"action": "ui", "id": "button:Arm"}},
    {"at": 10, "cmd": {"action": "ui", "id": "ux_sensor_combo", "value": 2}},
    {"when": {"vehicle": 1, "field": "alt_rel", "op": ">=", "value": 500},
      "timeout": 120,
      "cmd": {"action": "set_current_wp", "index": 3}},
    {"wait": 0, "cmd": {"action": "ui", "id": "checkbox:River", "value": true}}
  ]
}
```

트리거 (스텝마다 하나)

키	의미
<code>"wait": s</code>	이전 스텝 후 s초 대기
<code>"at": s</code>	시나리오 시작 후 s초 시점에 실행
<code>"when": {...}</code>	텔레메트리 조건 충족까지 대기 후 실행. <code>timeout</code> (초, 기본 300) 초과 시 그 스텝 건너뛴

조건 (when) 필드

`vehicle` (id), `field`, `op` (`>=` `<=` `>` `<` `==` `!=`), `value`. 사용 가능 `field`: `alt_rel`, `alt_amsl`, `groundspeed`, `heading`, `current_wp`, `lat`, `lon` (QGC가 45678 로 500ms마다 방송).

명령 (cmd) — 45679로 그대로 전송

형태	예
UI 컨트롤 조작	<code>{"action": "ui", "id": "<id>", "value": <값>}</code>
웨이포인트 N 이동	<code>{"action": "set_current_wp", "index": 3}</code>
활성 차량 선택	<code>{"action": "select_vehicle", "vehicleId": 4}</code>
가이드 액션	<code>{"action": "rtl"}</code> , <code>{"action": "takeoff", "altitude": 40}</code>
UxAS 패널	<code>{"action": "ux_publish_sar"}</code> , <code>{"action": "ux_river_mode", "mode": "center"}</code>

5. 대표 예시 — 조건부 SAR

"이륙 → 500m 도달하면 3번 웨이포인트로 이동 → 강 검색 켜기"

```
{
  "name": "조건부 강 검색",
```

```

"steps": [
  {"wait": 1, "cmd":{"action":"ui","id":"button:Arm"}},
  {"wait": 3, "cmd":{"action":"takeoff","altitude":60}},
  {"when":{"vehicle":1,"field":"alt_rel","op":>=,"value":500},
   "cmd":{"action":"set_current_wp","index":3}},
  {"wait": 0, "cmd":{"action":"ui","id":"ux_river_mode_combo","value":0}},
  {"wait": 0, "cmd":{"action":"ux_publish"}}
]
}

```

6. 녹화 → 재생 워크플로 (콘솔 하나로)

Monitor/Record ● 녹화 → QGC 조작 → 정지·Save .jsonl → Replay ▶ 재생 (또는 Scenario에서 Import→조건 추가 →Run)

- **Monitor/Record** 탭 — 45678 전 채널을 보며 .jsonl 로 녹화
- **Replay** 탭 — 그 파일을 원래 타이밍대로 재생(rate/필터)
- **Scenario** 탭 — 녹화 Import + 조건/시간 편집 후 실행, 라이브피드 더블클릭으로 스텝 추가

모두 `ui_scenario_studio.py` 한 프로그램 안에 있다. (독립 `eb_*` 스크립트도 그대로 남아 있어 CLI로 쓸 수 있다.)

실행 순서 — ① QGC 실행(EventBroadcaster 45678 / 45679 자동 개방) ② (조건 쓰려면) 차량 연결로 텔레메트리 방송 ③ `ui_scenario_studio.py` 실행. 조작하려는 컨트롤이 속한 뷰를 미리 한 번 열어 등록되게 할 것.